

Web Programming SE 3355 – Midterm Project

Title: Your Favourite Movie/Series Wiki

1. Objective

The purpose of this midterm project is to strengthen your understanding of frontend development with React and JSON-based data retrieval from web APIs. You will build a small but non-trivial Movie/Series Wiki.

Your application should consume a public movie/series-related API (for example, the Star Wars API – SWAPI – to build a Star Wars Wiki) and present the retrieved data in a clean, navigable, responsive and interactive interface built entirely with React and Tailwind.

2. Assignment Description

You are required to develop a Movie/Series Wiki web site that allows users to browse and explore information about a specific movie/series universe, collection, or theme. Examples:

- Star Wars wiki using SWAPI
- LOTR wiki using The One API
- GoT wiki using An API of Ice And Fire
- Star Trek wiki using STAPI
- Rick and Morty wiki using The Rick and Morty API
- Pokemon wiki using Pokéapi

The wiki should include listing pages, detail pages, and at least one search and filtering feature. All views and interactions must be implemented in React.

Minimum Requirements

1. Technology Stack & Project Setup

- The entire frontend must be implemented using React (Vite, Create React App, or a similar React build tool).
- Use function components and React Hooks (e.g., `useState`, `useEffect`).
- Organize your code under a clear structure (e.g., `src/components`, `src/pages`, `src/services`).
- Use Tailwind CSS for styling.

2. Data Source & API Integration

- Use at least one public web API as the main data source (e.g., SWAPI for Star Wars).
- Retrieve data using `fetch` or a similar library (e.g., `axios`).
- Implement proper loading states (e.g., “Loading...”) and error handling (e.g., “Failed to load data”).
- Use API responses to populate lists and detail pages (no hard-coded mock data for the main content).

3. Core Features

Your Movie Wiki should include at least:

a) Home Page:

- Brief introduction to your chosen universe/theme.
- Navigation links to main sections (e.g., Movies, Characters, Planets).

b) Listing Pages (at least 3):

- Example: list of movies/films, or characters, or planets/locations.
- Display multiple items in a grid or list layout (title, year, short info, etc.).
- Include basic pagination or “Load more” functionality if the API supports it or returns many items.

c) Detail Page:

- A separate view for a selected item (e.g., details of one movie or one character).
- Shows more detailed information (e.g., title, release date, director, cast, description, related items).
- Navigable via dynamic routing (e.g., /movies/:id)

d) Search and Filtering:

Implement at least one of:

- Text-based search (e.g., search movie/character by name), or
- Filter (e.g., filter movies by year, filter characters by species).

The UI should update reactively based on user input.

4. UI/UX and Navigation

a) Implement a navigation bar that allows users to move between the main sections (Home, Movies, Characters, etc.).

b) Use a client-side routing library (e.g., React Router).

c) Ensure a consistent visual style across pages (colors, typography, spacing).

d) Aim for a responsive layout, so the wiki is usable on different screen sizes.

5. State Management & Code Organization

a) Use React state (useState) and effects (useEffect) appropriately for fetching and managing data.

b) Pass data and callbacks using props; avoid deeply nested, tightly coupled components.

c) Separate concerns:

- API logic (e.g., api.js or services/api.js)
- UI components (e.g., MovieCard, CharacterList, SearchBar)

- Layout components (e.g., Navbar, Footer)

d) Write clean, readable code with meaningful variable/component names and comments where needed.

3. Submission Instructions

Please provide the link to your web application.

You may use Vercel for free deployment.

4. Evaluation Criteria

Your assignment will be graded according to the following rubric:

Criteria	Description	Weight
API Integration and Data Handling	Correct use of at least one public API, proper data fetching, handling of loading and error states, and meaningful use of API responses in the UI.	25%
React Component Architecture & State Management	Appropriate use of React components and hooks, clear separation of concerns, effective use of props and state, and avoidance of unnecessary duplication.	25%
Routing, Navigation & UI/UX Design	Clear and usable navigation between pages/views, intuitive layout, responsive design, and overall user experience quality.	20%
Code Quality and Project Structure	Clean, readable, and maintainable code; meaningful naming; sensible folder structure; presence of helpful comments.	20%
Creativity and Effort	Additional features beyond the minimum requirements, polish in design and user experience, and evidence of thorough testing and refinement.	10%

5. Deadline

- All submissions must be completed **before 14.11.2025**.
- No late submissions will be accepted or graded.

6. Notes and Academic Integrity

- This project must be implemented using React for the frontend.
- You must style your application using Tailwind CSS. Avoid using fully pre-built templates.
- You may refer to online documentation (e.g., React, MDN, API documentation) for syntax and examples.

- All code you submit must be written and understood by you.
- Do not copy code from other students, GitHub repositories, or tutorial projects without proper understanding and citation.
- All work must be your own. Any copied or shared files will be considered plagiarism. Any form of plagiarism or unauthorized collaboration will be handled according to university academic integrity policy.
- Do not use external templates or AI-generated code. Using of GenAI will negatively impact your grade.